QC SKILL · RECONSTRUCTION SPECIFICATION

qc-core

Shared laws, ledger schema 1.1, verdict math, .qc-profile.json, Discovery+Verify, and reexam. Load before every specialization.

Canonical source of truth is the **SKILL.md** tree in this repository. This PDF reconstructs that tree for porting. Do not treat this PDF as a second design. Do not vendor a specialization without **qc-core**.

Role

qc-core is the shared machinery for the QC suite. Specializations own ordered passes and what counts as a finding. This skill owns every invariant they would otherwise copy: the seven laws, ledger schema 1.1 (`qc-finding.schema.json`), verdict math in `scripts/qc.py`, the unified profile `.qc-profile.json`, Discovery+Verify, import-graph partitioning and the reexam set, deferred vocabulary, CLEAN lists, commits, and suite order.

A standalone port is **this tree plus one specialization**. A suite port is this tree plus the specializations plus **qc-all**. Installing a specialization without qc-core is an incomplete port.

The seven laws

Do not weaken these. Full text: skills/qc-core/references/laws.md.

#	Law
1	Harden what exists. No features, no architecture, no behavior change. Do not expand the audit surface.
2	Concrete failure scenario or it is not a finding. Deep / P1+ findings carry trigger, violated_invariant, observable.
3	Fix in place or defer (fixed: false + closed-vocabulary deferred_because).
4	P0 blocks even if fixed (suite-level human-review gate).
5	Isolated commits. Batch clean units, never skip execution.
6	Mechanical before deep. Do not collapse the 14 hardening passes or reverse the order.
7	Dual-vote to fix. Both votes say real → fix; otherwise defer. Never silently drop.

North star: three clean passes is the expected output of clean code, not a reason to manufacture findings. Why the deep passes exist: catastrophe is multi-factor; latent failures remain after mechanical passes; there is no isolated root cause; change creates new combinations with pieces that did not themselves change; invariants are continuously created; system as imagined is not system as found. Clean Carmack across all modules plus a green Chaos suite remains the expected terminal state.

Ledger

Write `.qc-findings/<skill>.json` as one JSON object conforming to `skills/qc-core/references/qc-finding.schema.json`. Schema version **1.1**. qc-core owns this schema. Do not copy a fork into a specialization. Do not treat qc-all as the schema owner. Atomic write: temp file in the same directory, then os.replace().

Required finding fields: id, severity, file, what, fixed. Confidence on new findings: mechanical | pattern | proven | human. Deep / P1+ (non-mechanical) findings need scenario.{trigger, violated_invariant, observable}. For Carmack, Chaos, Semantic, and Architectural findings, trigger names a combination (second step, second implementation, second location, or dual path).

Verdict math

One algorithm: scripts/qc.py::compute_verdict. Do not reimplement in a skill, a shell script, or a prompt.

Verdict	Condition
NOT_READY	Open P1 (fixed:false, no deferred_because). Presence P0 policy (hardening): any P0, fixed or not. Open P0 policy (packaging, docs, coherence): only an open P0.
READY_WITH_DEBT	No blocking P0/open P1; at least one P1 (fixed or deferred) or an open P2.
READY	No P0 (under policy), no P1; P2/P3 clean or deferred.
SKIPPED	Reserved for qc-all marking a skill it did not run.

P3 never blocks. verify_ledger.py recomputes this and fails on mismatch.

Suite algebra (scripts/suite_rollup.py)

- Legal order: qc-packaging → qc-hardening → qc-coherence → qc-docs.
- Any skill verdict NOT_READY → suite NOT_READY.
- Any P0 in any non-skipped ledger (fixed or not) → suite NOT_READY (human-gate).
- Else any skill READY_WITH_DEBT, or any deferred P1 → suite READY_WITH_DEBT.
- Else READY. All-skipped / empty → READY.

Unified profile

Source of truth: `.qc-profile.json` at the target repo root, schema `qc-profile.schema.json`. Every qc-* skill reads and writes it. Human summary `.qc-profile.md` is generated by scripts/render_profile.py and must not be edited by hand.

Section	Role
examination	Per skill, per module/symbol: EXAMINED / FINDING / NOT_YET, plus invariant or Carmack 5Q.
pass_streaks	Consecutive clean runs; drives hardening quick-check eligibility.
hot_spots	Repeat-offender artifacts.
pass_debt	Deep-pass hits a mechanical pass should have caught (why_missed names the missed combination).
deferred	Closed-vocabulary deferred findings, visible to every later skill.
invariants	Living registry harvested from Carmack, must/never comments, and tests. Coherence enforces it.
canonical_forms	Semantic ground truth: concept → canonical name + aliases.
layer_model	Architectural ground truth: layers + permitted import direction.
truth_map	Docs claims → evidence pointer (file:line, test name, or user-provided:<id>).
operating_facts	User-provided docs facts with timestamps (source: user-provided).
clusters	Machine-checkable clustering/responsibility judgments.

For qc-hardening, profile examination.qc-hardening is the Carmack manifest. An EXAMINED row requires the five Carmack questions or {invariant, assumptions, sequence_risk}. A one-sentence invariant is incomplete. verify_profile.py rejects boilerplate.

Discovery+Verify and reexam

Shared workflow: `skills/qc-core/workflows/discovery-verify.js`. kind=groups (hardening) or kind=lenses (coherence). Remediation stays with the caller; the workflow never edits or commits.

Partitioning: import graph first, then language boundary, then LOC pack (500–800). Partial-rerun uses the reexam set: changed files plus import-graph neighbors that already sit in hot_spots[] or pass_debt[].

```
python3 skills/qc-core/scripts/partition.py --reexam --since <sha> --profile .qc-profile.json --json
```

Dual-vote: both votes say real → fix; split or any uncertain → defer; never silently drop. mechanical and human confidence skip verification.

Commands

From this repository: python3 skills/qc-core/scripts/<script>.py --help. After install, the same scripts live in the skill tree (for example ~/.claude/skills/qc-core/scripts/), not inside the target repo. Call that copy against the target's .qc-findings/ and .qc-profile.json.

```
python3 skills/qc-core/scripts/verify_ledger.py --help
```

```
python3 skills/qc-core/scripts/verify_profile.py --help
```

```
python3 skills/qc-core/scripts/suite_rollup.py --help
```

```
python3 skills/qc-core/scripts/verify_port.py --help
```

Port acceptance is not --help. After a run produces a ledger, recover the seeded defects: --expected skills/qc-core/fixtures/hardening/expected.json (also fixtures/coherence/ and fixtures/docs/) --ledger path/to/ledger.json. Packaging has no seeded fixture. A reconstruction is faithful only when those three recoveries succeed at the declared severities and clean modules are EXAMINED.

Suite order

qc-packaging → **qc-hardening** → **qc-coherence** → **qc-docs**. Packaging wraps what exists. Hardening drives residual runtime failure scenarios to zero and harvests invariants. Coherence consumes that ledger. Docs run last so the truth-map matches the system that survived.